

06026240 INTELLIGENT SYSTEM DEVELOPMENT

Chapter VIII

Evaluation and Overfitting

with Dr. Bhattarabhorn Wattanacheep

- เลือก metric ให้ตรงกับชนิดของงาน และอธิบายได้ว่าทำไมไม่ใช้ตัวอื่น
- อ่านค่า metric ออกว่า "ค่าสูงแปลว่าอะไร ค่าต่ำแปลว่าอะไร" ในบริบทของงานตัวเอง — ไม่ใช่แค่ว่าสูงดีต่ำแย่
- ตรวจได้ว่าโมเดลของตัวเองกำลัง overfit อยู่หรือเปล่า และถ้าใช่ จะแก้ด้วยวิธีไหน
- รู้จัก metric ที่วิชานี้ไม่ได้ใช้
- รันสคริปต์ประเมินผลกับงานโปรเจกต์ของตัวเอง แล้วเอาตัวเลขไปใส่รายงานได้

1

งานจำแนก — ผิดได้ก็แบบ และแต่ละแบบเจ็บไม่เท่ากัน

ส่วนที่ 1 · confusion matrix และทุกอย่างที่งอกออกมาจากมัน

ตารางสี่ช่องที่เป็นต้นทางของทุกอย่าง

confusion matrix — ตัวอย่าง งานตรวจจับหน้า transcript ที่สแกนไม่ผ่านเกณฑ์

positive class = "ไม่ผ่าน" (คือสิ่งที่ระบบพยายามจับ = ใบที่ไม่ควรส่งต่อ)

	เฉลยจริง: ไม่ผ่าน	เฉลยจริง: ผ่าน	
ทำนายว่า ไม่ผ่าน	TP จับได้ถูก	FP ตีตกใบที่ใช้ได้	→ Precision = TP / (TP+FP)
ทำนายว่า ผ่าน	FN ปล่อยหลุด	TN ปล่อยถูก	
	↑ Recall = TP / (TP+FN)		

FP = ตีตกใบที่จริง ๆ ใช้ได้ → เสียเวลาสแกนใหม่ | FN = ปล่อยใบเสียเข้าระบบ → ข้อมูลผิดเข้าฐานข้อมูล

ทุกตัวคำนวณจากตารางสี่ช่องเมื่อกี้ ค่าอยู่ระหว่าง 0 ถึง 1

positive class = "ไม่ผ่าน" (คือสิ่งที่ระบบพยายามจับ = ใบที่ไม่ควรส่งต่อ)

metric	สูตร	ค่าสูงแปลว่า	ค่าต่ำแปลว่า
Accuracy	$(TP+TN) / \text{ทั้งหมด}$	ตัดสินใจทั้งสองฝั่งโดยรวม	ผิดพลาด แต่ไม่รู้ว่าผิดของดี หรือปล่อยของเสียหลุด
Precision	$TP / (TP+FP)$ — จากใบที่ตีตก	ตีตกใบไหน ใบนั้นเบลอจริง	ในกองที่ตีตก มีใบใช้ได้ปนเยอะ user เสียเวลาสแกนใหม่
Recall	$TP / (TP+FN)$ — จากใบที่เบลอจริง	ใบเบลอถูกจับได้เกือบครบ ไม่หลุดเข้า OCR	ใบเบลอหลุดเข้า OCR เยอะ ข้อมูลผิดเข้าฐานข้อมูล
Specificity	$TN / (TN+FP)$ — จากใบที่ชัดจริง	ใบที่ชัดถูกปล่อยผ่านเกือบครบ ไม่โดนตีตกทั้ง	ตีตกใบที่ใช้ได้เยอะ เพิ่มงานสแกนซ้ำให้ user
F1	ค่าเฉลี่ยฮาร์โมนิก ของ P กับ R	P กับ R ดีทั้งคู่ ไม่เอียงข้างใดข้างหนึ่ง	ด้านหนึ่งแยจนคูณรวม ต้องแยกดู P กับ R

เลือกข้างให้ถูก — เมื่อไร Precision สำคัญกว่า เมื่อไร Recall สำคัญกว่า



สองอย่างนี้แลกกันเสมอ ดันตัวหนึ่งขึ้น อีกตัวจะลง ต้องเลือกตามราคาของความผิดพลาด

เอา Precision ก่อน — เมื่อการเตือนผิดมีราคาแพง

- กรองใบเบลอก่อนเข้า OCR — ตีตกใบที่ใช้ได้ user ต้องสแกนซ้ำจนเลิกเชื่อระบบ
- แจ้งเตือน transcript ต้องสงสัยว่าปลอม แล้วส่งให้นิติกร — เตือนผิดบ่อย เสียความสัมพันธ์กับนักศึกษา
- ระบบตอบคำถามหลักสูตร — ตอบมั่วหนึ่งครั้ง ความน่าเชื่อถือหายทั้งระบบ
- อาการเมื่อ Precision ต่ำ คนใช้งานจะเริ่มกด ignore ทุกการเตือน เรียกว่า alarm fatigue

เอา Recall ก่อน — เมื่อการปล่อยหลุดมีราคาแพง

- กรองใบเบลอก่อนเข้า OCR — ปล่อยใบเบลอหลุด เข้าระบบ
- ตรวจสอบวิชาที่ยังไม่ถูกสกัดจากเล่มหลักสูตร — ตกหล่นหนึ่งวิชา ระบบตอบว่า “ไม่มีวิชานี้”
- คัดกรองโรคจากภาพเอกซเรย์ — พลาดผู้ป่วยจริงหนึ่งคน แพงกว่าเรียกมาตรวจซ้ำสิบคน
- อาการเมื่อ Recall ต่ำ ระบบดูเงียบเรียบร้อยดี เพราะปัญหาไม่เคยถูกรายงาน

ชุดข้อมูล 1,000 หน้า มีหน้าที่ปลอมแปลงอยู่ 10 หน้า

99.0%

Accuracy ของโมเดลที่ตอบว่า "ไม่ปลอม" ทุกหน้า

0.0

Recall ของโมเดลตัวเดียวกัน — จับของปลอมไม่ได้
สักหน้า

0.0

MCC — ยืนยันว่าโมเดลไม่ได้เรียนรู้อะไรเลย

accuracy 99% ที่นี้ไม่ได้แปลว่าโมเดลเก่ง แต่แปลว่าข้อมูลไม่สมดุล 99 ต่อ 1

เมื่อไรที่คลาสหนึ่งมีสัดส่วนเกิน 80% ให้เลิกดู accuracy แล้วไปดู Recall ของคลาสหายาก, MCC แทน

MCC (Matthews Correlation Coefficient) คือค่าสหสัมพันธ์
ระหว่างคำทำนายกับเฉลยจริง

$$MCC = (TP \cdot TN - FP \cdot FN) / \sqrt{((TP+FP)(TP+FN)(TN+FP)(TN+FN))}$$

+1 = ทำนายตรงเฉลยทุกใบ

0 = พอๆกับเดาสุ่ม

-1 = ทำนายกลับด้านทุกใบ

2

งานทำนายค่าต่อเนื่อง — วัต่ว่าพลาดไปเท่าไร

ส่วนที่ 2 · regression, depth estimation และการนับหน่วยกิต

metric สำหรับค่าต่อเนื่อง — ทุกตัวยิ่งต่ำยิ่งดี ยกเว้น R square



ตรงข้ามกับกลุ่มแรก กลุ่มนี้วัด "ความผิดพลาด" ไม่ใช่ "ความถูกต้อง"

metric	วัดอะไร	ค่าสูงแปลว่า	ค่าต่ำแปลว่า / ข้อควรระวัง
MAE	ค่าเฉลี่ยของขนาดความผิดพลาด	พลาดเยอะโดยเฉลี่ย อ่านง่ายที่สุด มีหน่วยเดียวกับข้อมูล	พลาดน้อย ไม่ไวต่อค่าผิดปกติ อาจซ่อนเคสพังหนัก
MSE / RMSE	ยกกำลังสองก่อนเฉลี่ย RMSE ถอดรากกลับมาเป็นหน่วยเดิม	มีเคสที่พลาดหนักอยู่ แม้จะไม่กี่เคส	ไม่มีเคสพังหนัก ใช้เมื่อความผิดพลาดใหญ่ต้องถูกลงโทษหนัก
MAPE	ความผิดพลาดเป็นเปอร์เซ็นต์	พลาดมากเมื่อเทียบกับขนาดของค่าจริง	ระเบิดเป็นอนันต์เมื่อค่าจริงเป็นศูนย์ ห้ามใช้เมื่อค่าจริงเข้าใกล้ 0
Huber loss	ลูกผสม MAE กับ MSE	เหมือน MSE ในช่วงพลาดน้อย เหมือน MAE เมื่อพลาดมาก	ใช้เป็น loss ตอนเทรน เมื่อข้อมูลมี outlier ปนอยู่

Mean Absolute Error · มีหน่วยเดียวกับข้อมูล · ค่ายิ่งต่ำยิ่งดี

สูตร

$$MAE = (1/n) \sum |y_i - \hat{y}_i|$$

n = จำนวนจุดที่วัดในชุดทดสอบ

y_i = ระยะจริงจากเซนเซอร์วัดระยะ

$\hat{y}_i$ = ระยะที่โมเดลทำนายจากภาพ

$|\dots|$ = ค่าสัมบูรณ์ ทำให้ทำนายใกล้เคียงกับใกล้เคียงนับเท่ากัน

ตัวอย่าง 3 จุด ที่มีค่าความผิดพลาดต่าง 0.1, 0.2 และ 0.6 ม.

$$MAE = (0.1 + 0.2 + 0.6) / 3 = 0.3 \text{ ม.}$$

อ่านค่ายังไง

- $MAE = 0.15$ ม. แปลว่าเฉลี่ยแล้วโมเดลบอกระยะพลาดไป 15 ซม.
- ทุกจุดน้ำหนักเท่ากัน พลาด 2 ม. ที่ขอบวัตถุถูกเฉลี่ยจนแทบไม่เห็น
- ระวัง: ค่าดูดีได้ทั้งที่ขอบวัตถุทั้งหมด ต้องอ่านคู่กับ RMSE
- เหมาะใช้รายงานกับคนนอกทีม เพราะเทียบกับระยะจริงได้ทันที
- MAE ต่ำแต่ภาพความลึกดูเพี้ยน ให้ดูการกระจายของความพลาด ไม่ใช่ค่าเฉลี่ย

Mean Squared Error และรากที่สองของมัน · RMSE มีหน่วยเดียวกับข้อมูล · ค่ายิ่งต่ำยิ่งดี

สูตร

$$\text{MSE} = (1/n) \sum (y_i - \hat{y}_i)^2$$

$$\text{RMSE} = \sqrt{\text{MSE}}$$

ยกกำลังสองก่อนเฉลี่ย ความพลาดใหญ่จึงถูกขยาย
พลาด 2 ม. หนักกว่าพลาด 0.2 ม. ถึง 100 เท่า

อ่านค่ายังไง

- RMSE ไกล MAE แปลว่าความพลาดกระจายสม่ำเสมอทั้งภาพ
- RMSE สูงกว่า MAE มาก แปลว่ามีไม่กี่บริเวณที่พลาดหนักลากค่าขึ้น
- เช่น MAE 0.15 ม. แต่ RMSE 0.9 ม. คือขอบวัตถุหรือผิวสะท้อนแสงพัง
- ใช้เมื่อพลาดหนักจุดเดียวแพงกว่าพลาดเล็กน้อยหลายจุด เช่น หุ่นยนต์ชนของ
- ใช้เป็นตัวจับบริเวณพังหนัก ไม่ใช่ตัวรายงานให้คนนอกฟัง

Mean Absolute Percentage Error · ไม่มีหน่วย เทียบข้ามสเกลได้ · ค่ายิ่งต่ำยิ่งดี

สูตร

$$\text{MAPE} = (100/n) \sum |(y_i - \hat{y}_i) / y_i|$$

หารด้วยระยะจริงก่อนเฉลี่ย จึงได้สัดส่วน ไม่ใช่ขนาด

ผลลัพธ์เป็นเปอร์เซ็นต์ ไม่มีหน่วยเมตรติดมา

y_i ใกล้ 0 ทำให้ตัวหารเล็กจนค่าระเบิด

ทำนายใกล้เกินพลาดได้มากที่สุด 100% แต่ไกลเกินไม่มีเพดาน

อ่านค่ายังไง

- MAPE 5% แปลว่าเฉลี่ยแล้วพลาดไป 5% ของระยะจริงในแต่ละจุด
- ข้อดี: พลาด 0.5 ม. ที่ระยะ 2 ม. น้อยกว่าพลาด 0.5 ม. ที่ระยะ 50 ม.
- จึงเทียบฉากใกล้กับฉากไกลในชุดข้อมูลเดียวกันได้
- ห้ามใช้กับจุดที่ระยะจริงเข้าใกล้ 0 เช่น วัตถุชิดหน้ากล้อง
- ลำเอียง: ลงโทษการทำนายไกลเกินน้อยกว่าการทำนายใกล้เกิน

Smooth L1 · เป็น loss function ไม่ใช่ตัวรายงานผล · ค่ายิ่งต่ำยิ่งดี

สูตร

$$e = y_i - \hat{y}_i$$

$$L = \frac{1}{2}e^2 \quad \text{ถ้า } |e| \leq \delta$$

$$L = \delta|e| - \frac{1}{2}\delta^2 \quad \text{ถ้า } |e| > \delta$$

เดลตา คือจุดตัดที่เราเลือกเอง ว่าพลาดเท่าไรถือว่ามาก
พลาดต่ำกว่าเดลตา คิดแบบ MSE สูงกว่านั้น คิดแบบ MAE
ที่ลบท้ายสูตร เพื่อให้สองช่วงต่อกันสนิท ไม่มีรอยหัก

อ่านค่ายังไง

- ช่วงพลาดน้อย โค้งเหมือน MSE จึงไล่ให้แม่นยำได้ละเอียด
- ช่วงพลาดมาก เป็นเส้นตรงเหมือน MAE จุดพังหนักจึงไม่ลากโมเดลทั้งตัว
- เดลตาเล็ก = พฤติกรรมใกล้เคียง MAE เดลตาใหญ่ = ใกล้เคียง MSE
- เหมาะกับเฉลี่ยที่มีค่าผิดปกติ เช่น เซนเซอร์วัดระยะอ่านพลาดที่ขอบวัตถุ
- เป็นตัวคุมการเทรน เวลารายงานผลยังใช้ MAE กับ RMSE เหมือนเดิม

4

งานข้อความและ OCR — วัดว่าอ่านผิดกี่ตัว

ส่วนที่ 4 · CER, WER และกับดักของภาษาไทย

ทั้งคู่มารูจากระยะแก้ไข (edit distance) — นับว่าต้องแก้กี่ครั้งถึงจะตรงเฉลย

$CER = (S + D + I) / N$ นับหน่วยเป็นตัวอักษร $WER = (S + D + I) / N$ นับหน่วยเป็นคำ

S = แทนที่ D = ลบ I = แทรก N = จำนวนหน่วยในเฉลย

เฉลย : 06026240 การพัฒนาระบบอัจฉริยะ 3(2-2-5) N = 42 ตัวอักษร / 5 คำ

ตัดคำได้ : [06026240] [การพัฒนา] [ระบบ] [อัจฉริยะ] [3(2-2-5)]

กรณี A ที่อ่านได้: 0602624O การพัฒนาระบบอัจฉริยะ 3(2-2-5) อ่าน O เป็น O หนึ่งตัว

$CER = 1/42 = 2.4\%$ $WER = 1/5 = 20\%$ ผิดตัวเดียว ทั้งคำนั้นผิดหมด

กรณี B ที่อ่านได้: 0602624O การพัฒนาอัจฉริยะ 3(2-2-5) กรณี A + อ่านตกคำ ระบบ

$CER = 5/42 = 11.9\%$ $WER = 2/5 = 40\%$ ตกคำเดียว = ตกไป 4 ตัวอักษร

ภาษาอังกฤษตัดคำด้วยช่องว่าง -> ง่าย ภาษาไทยไม่มีช่องว่าง -> ต้องใช้ตัวตัดคำ เช่น `pythainlp.word_tokenize`

CER 2.4% ฟังดูดีมาก แต่ WER 20% ตรงกับความเจ็บจริงกว่า เพราะหน่วยที่เจ้าหน้าที่ต้องแก้คือรหัสวิชาทั้งตัว ไม่ใช่ตัวอักษรเดียว

5

งาน LLM — วัสดุที่ไม่มีคำตอบเดียว

ส่วนที่ 5 : โครงสร้าง JSON, การค้นคืน, การตอบคำถาม และ NL to SQL

วิธีคิดของทั้งส่วนนี้

อย่าวัด "คำตอบดีไหม" แต่วัดที่ละชั้นว่าฟังตรงไหน

ระบบตอบคำถามหลักสูตรมีสี่ชั้น — สกัด, ตรวจสอบ schema, คั่นคั่น, ตอบ
ถ้าวัดแค่ปลายทาง เวลาตอบผิดเราจะไม่รู้เลยว่าต้องไปแก้ชั้นไหน

ใช้กับแล็บ 8A และ 8B ตรงจุดที่ LLM ต้องคายผลออกมาเป็น JSON

metric	วัดอะไร	ค่าสูงแปลว่า	ค่าต่ำแปลว่า
JSON valid rate	สัดส่วนผลลัพธ์ที่ parse เป็น JSON ได้	โมเดลคายรูปแบบได้เสถียร	โมเดลพ่นข้อความอธิบายปมมา แก้ด้วย prompt หรือ structured output
Schema pass rate (ก่อนซ่อม)	สัดส่วนที่ผ่าน Pydantic ตั้งแต่รอบแรก	โมเดลเข้าใจ schema ดี ต้นทุนต่ำเพราะไม่ต้องเรียกซ้ำ	ต้องพึ่ง repair loop มาก แปลว่า prompt หรือ schema ยังกำกวม
Repair rate	สัดส่วนที่ต้องวนซ่อม และซ่อมกี่รอบ	ค่าใช้จ่ายและเวลาสูงขึ้น แต่ยังกู้ได้	ถ้าซ่อมแล้วยังไม่ผ่าน = ต้อง abstain อย่าปล่อยข้อมูลที่ไม่ผ่านลงฐานข้อมูล
Field-level Precision / Recall	วัดรายฟิลด์ ไม่ใช่รายเอกสาร	Recall สูง = สกัดครบไม่ตกหล่น Precision สูง = ไม่เติมของที่ไม่มี	Recall ต่ำ = วิชหายไปจากฐานข้อมูล Precision ต่ำ = มีวิชาฟิอยู่ในระบบ
Coverage / Abstain rate	สัดส่วนที่ระบบยอมตอบ เทียบกับที่ยอมแพ้	ตอบเยอะ แต่ต้องดูคู่กับความแม่นยำ ตอบ 100% แล้วผิดครึ่ง = แย่กว่าตอบ 60% ที่ถูกต้อง	ยอมแพ้น้อย งานตกไปที่คน แต่ปลอดภัยกว่าเดาแล้ว

วัดคำตอบสุดท้ายและความเชื่อสัตย์ต่อแหล่งข้อมูล



สามตัวบนวัดความถูกต้อง สามตัวล่างวัดว่าคำตอบมาจากเอกสารจริงหรือแต่งเอง

metric	วัดอะไร	ค่าสูงแปลว่า	ค่าต่ำแปลว่า
Exact Match	คำตอบตรงเฉลยแบบเป๊ะ	ตอบถูกเป๊ะ เหมือนกับคำถามที่มี คำตอบเดียว เช่น จำนวนหน่วยกิต	อาจตอบถูกแต่เขียนต่างจากเฉลย จึงต้องดู token-F1 ประกอบ
Token F1	ความทับซ้อนระดับคำระหว่างคำตอบกับเฉลย	สาระตรงกัน แม้อ้อยคำต่าง	ต่ำทั้งที่ EM สูง เป็นไปได้ยาก ถ้า F1 ต่ำมากคือตอบคนละเรื่อง
Execution Accuracy (SQL)	รัน SQL ที่ไม่เคลเขียนแล้วผลลัพธ์ตรงกับที่ควรได้	ระบบ NL to SQL ใช้งานได้จริง เป็นตัวที่สำคัญที่สุดของกลุ่ม B	ต่ำ = แปลคำถามเป็น SQL ผิด ต้องดูว่าผิดที่ WHERE หรือที่ JOIN
Valid SQL rate	สัดส่วน SQL ที่รันผ่านโดยไม่ error	ไวยากรณ์ถูก แต่ยังไม่รับประกันว่าตอบตรงคำถาม	ต่ำ = ต้องรัดกุมเรื่อง schema ใน prompt หรือใช้ few-shot เพิ่ม
Faithfulness / Groundedness	ทุกประโยคในคำตอบมีที่มาจากบริบทที่ให้ไปจริงไหม	คำตอบยึดกับเอกสาร ตรวจสอบย้อนได้	ต่ำ = แต่งเรื่อง แม้คำตอบจะฟังดูถูก คือความเสี่ยงอันดับหนึ่งของ RAG
Citation coverage	สัดส่วนคำตอบที่ระบุหน้าหรือแถวในฐานข้อมูลอ้างอิงได้	ผู้ใช้ตรวจสอบเองได้ เพิ่มความน่าเชื่อถือมหาศาล	ต่ำ = ผู้ใช้ต้องเชื่อเราอย่างเดียว ซึ่งไม่ควรเกิดในระบบข้อมูลราชการ

วิธีที่นิยมมากในอุตสาหกรรมตอนนี้ แต่มีข้อควรระวังที่ต้องรู้ก่อนใช้

ใช้ได้ดีเมื่อ

- คำตอบที่ถูกมีได้หลายแบบ จนเขียนเฉลยตายตัวไม่ไหว เช่น การอธิบายเนื้อหาวิชา
- ใช้เป็นตัวคัดกรองขั้นแรก เพื่อลดจำนวนเคสที่คนต้องตรวจสอบ
- ให้เกณฑ์ที่ชัดเจนเป็นข้อ ๆ และให้ตอบเป็นคะแนนพร้อมเหตุผล ไม่ใช่ให้ตอบดี/ไม่ดีลย ๆ
- มีการตรวจสอบเทียบกับคนอย่างน้อย 50 เคส แล้วรายงาน Cohen kappa ระหว่างคนกับ LLM

อย่าเชื่อสนิทเพราะ

- ลำเอียงเข้าข้างคำตอบยาวและคำตอบที่เขียนสละสลวย แม้เนื้อหาจะผิด
- ลำเอียงตามลำดับที่ให้เปรียบเทียบ สลับ A กับ B แล้วคำตอบอาจเปลี่ยน
- ลำเอียงเข้าข้างคำตอบที่ไม่เคลอะระกูลเดียวกันเขียน
- ไม่เสถียร — ถามคำถามเดิมสองครั้งอาจได้คะแนนไม่เท่ากัน ต้องตั้ง temperature เป็น 0 และรันซ้ำ
- ห้ามใช้เป็นตัวเลขเดียวในรายงาน ต้องมี metric ที่คำนวณตรงจากเฉลยประกอบเสมอ

8

Overfitting — เมื่อโมเดลจำข้อสอบเก่าได้ แต่ทำข้อสอบใหม่ไม่ได้

ส่วนที่ 8 - คืออะไร ดูยังไง แก้อย่างไร และรูปแบบที่ซ่อนอยู่ในงาน LLM

นิยามที่ง่ายที่สุด

Overfitting คือการจำ แทนที่จะเป็นการเรียนรู้

นักศึกษาที่ท่องข้อสอบเก่าห้าปีย้อนหลังจนได้เต็ม แต่พอเจอโจทย์ที่เปลี่ยนตัวเลขก็ทำไม่ได้
โมเดลก็เหมือนกัน — มันจำรายละเอียดเฉพาะของชุดฝึก รวมทั้งสิ่งที่เป็นความบังเอิญและ noise

ดูจากความสัมพันธ์ระหว่างผลบนชุดฝึกกับผลบนชุดตรวจสอบ

A

Underfitting

ชุดฝึกก็แย่ ชุดตรวจสอบก็แย่

โมเดลง่ายเกินไป หรือฝึกน้อยเกินไป
หรือ feature ไม่พอ

สัญญาณ: train accuracy ต่ำตั้งแต่
แรกและไม่ขยับ

แก้: เพิ่มขนาดโมเดล ฝึกนานขึ้น ลด
regularization

B

พอดี

ชุดฝึกดี ชุดตรวจสอบดี และห่างกันไม่มาก

ช่องว่างระหว่างสองค่าประมาณ 2 ถึง 5
จุดถือว่าปกติ

สัญญาณ: กราฟทั้งสองเส้นลงไปด้วยกัน
แล้วนิ่ง

นี่คือจุดที่ควรหยุดฝึกและบันทึกโมเดล

C

Overfitting

ชุดฝึกดีมาก แต่ชุดตรวจสอบแย่

ช่องว่างกว้างขึ้นเรื่อย ๆ เมื่อฝึกต่อ

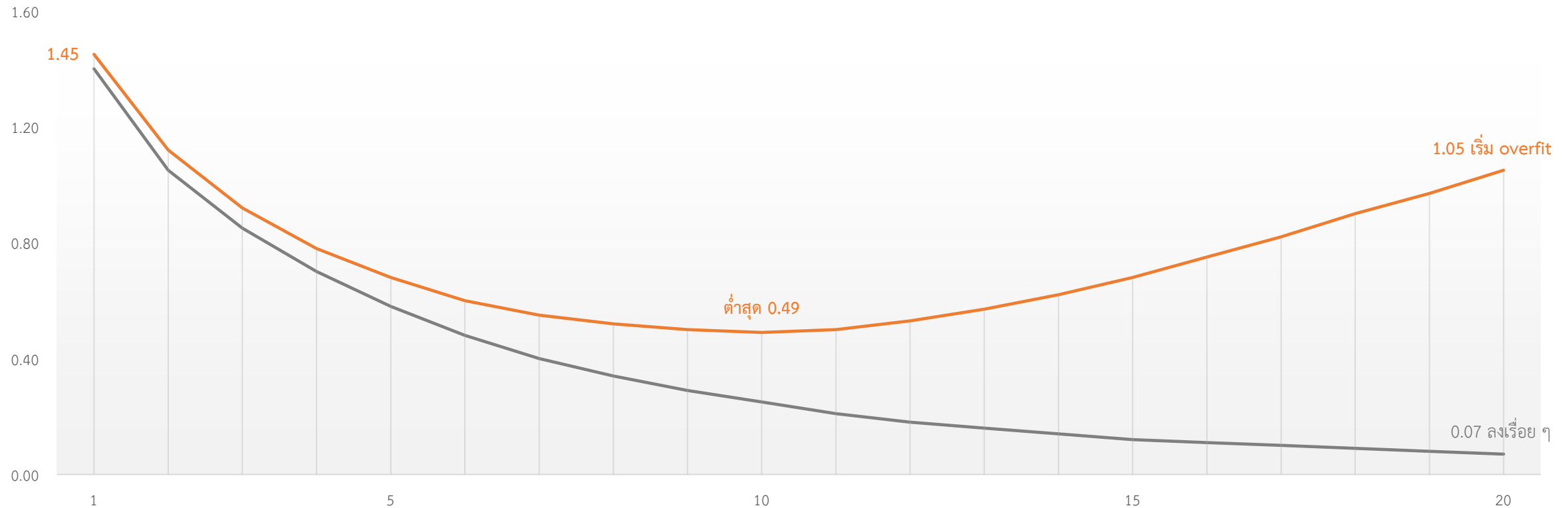
สัญญาณ: train loss ยังลง แต่ val
loss เริ่มขึ้น

แก้: หยุดฝึกที่จุดต่ำสุดของ val loss
เพิ่มข้อมูล เพิ่ม regularization

กราฟที่ต้องดูทุกครั้งที่ฝึกโมเดล



learning curve — เส้น train กับ validation ในกราฟเดียวกัน



train loss ที่ลดลงเรื่อย ๆ ไม่ใช่ข่าวดี — มันลดได้ตลอดถ้าโมเดลใหญ่พอที่จะจำข้อมูลทั้งหมด

สัญญาณที่บอกว่ากำลัง overfit — และดูจากไหน

ไม่จำเป็นต้องมีทุกข้อ เจอข้อเดียวก็ควรสงสัยแล้ว

สัญญาณ	ดูจากไหน	ความหมาย
val loss เริ่มขึ้น ขณะที่ train loss ยังลง	learning curve	จุดตัดคือจุดที่ควรหยุด ทุก epoch หลังจากนั้นคือการถอยหลัง
ผลดีมากบนชุดทดสอบ แต่พังกับข้อมูลจริง	ทดลองกับหน้าเอกสารใหม่ ที่เพิ่งสแกนมา	มักแปลว่าชุดทดสอบไม่ได้เป็นตัวแทนของข้อมูลจริง
ผลแกว่งมากเมื่อเปลี่ยน seed หรือเปลี่ยนวิธีแบ่ง	รันซ้ำ 3 รอบด้วย seed ต่างกัน	โมเดลไม่เสถียร มักเกิดเมื่อข้อมูลน้อยเกินไป ตัวเลขที่รายงานจึงเป็นความบังเอิญ
เพิ่มพารามิเตอร์แล้ว ผล val แย่ลง	เทียบโมเดลเล็กกับโมเดลใหญ่	ความจุของโมเดลเกินปริมาณข้อมูลที่มี
โมเดลแม่นยำเฉพาะกับ เอกสารจากเครื่องสแกนเดิม	slice analysis แยกตาม แหล่งที่มา	โมเดลเรียนรู้ลักษณะเฉพาะของเครื่องสแกน ไม่ใช่เรียนรู้ตัวอักษร — เป็น shortcut learning

วิธีแก้ overfitting — เรียงตามลำดับที่ควรลองก่อนหลัง



ข้อ 1 ถึง 3 ให้ผลมากที่สุดเสมอ ข้อหลัง ๆ คือการปรับจูน

วิธี	ทำอะไร	เหมาะกับสถานการณ์
1. เพิ่มข้อมูล	เก็บตัวอย่างจริงเพิ่ม โดยเฉพาะจากแหล่งที่ต่างจากเดิม	ได้ผลดีที่สุดเสมอ แต่แพงที่สุด
2. Data augmentation	หมุน ปรับแสง ใส่ noise แบบที่เกิดจริง	เมื่อเก็บข้อมูลเพิ่มไม่ได้ แต่ต้องเป็นการแปลงที่สมจริง
3. Early stopping	หยุดฝึกที่จุดที่ val loss ต่ำสุด พร้อมตั้ง patience ประมาณ 5 ถึง 10 epoch	ทุกกรณี แทบไม่มีเหตุผลที่จะไม่ใช้
4. Transfer learning	ใช้โมเดลที่ฝึกมาแล้ว แช่แข็งชั้นต้น ฝึกเฉพาะชั้นท้าย	ข้อมูลน้อยกว่าหนึ่งพันตัวอย่าง คือกรณีของคุณเกือบทั้งหมด
5. ลดขนาดโมเดล	ใช้ ResNet18 แทน ResNet50 ลดจำนวนชั้นหรือจำนวนหน่วย	เมื่อจำนวนพารามิเตอร์มากกว่าจำนวนตัวอย่างมาก
6. Regularization L1 / L2	เพิ่มค่าปรับตามขนาดของน้ำหนัก L1 ยังช่วยคัด feature ที่ไม่ดีด้วย	งานตารางข้อมูลและโมเดลเชิงเส้น
7. Dropout	สุ่มปิดหน่วยประสาทระหว่างฝึก โดยทั่วไป 0.2 ถึง 0.5	โครงข่ายที่มีชั้น fully connected ใหญ่ ๆ
8. Label smoothing	เปลี่ยนเฉลยจาก 1.0 เป็น 0.9 ไม่ให้โมเดลมั่นใจเกินไป	งานจำแนกที่ป้ายกำกับมี noise ปนอยู่
9. Cross-validation	ใช้ k-fold เพื่อให้ค่าที่รายงานเสถียร	ข้อมูลน้อยจนแบ่ง val ที่เดียวแล้วผลกว้าง
10. Ensemble	รวมผลจากหลายโมเดลหรือหลาย seed	เมื่อยอมแลกความเร็วกับความเสถียร

แลกความเร็วและความซับซ้อนกับความเสถียร — เวลา inference เพิ่มขึ้นตามจำนวนโมเดล

1. Bagging — ลดความแปรปรวน

- สุ่มข้อมูลแบบ bootstrap ฝึกหลายโมเดลขนานกัน แล้วโหวตหรือเฉลี่ย
- ตัวอย่าง Random Forest
- ใช้เมื่อโมเดลเดี่ยวกว้างตาม seed หรือตามวิธีแบ่งข้อมูล



2. Boosting — ลดความเอนเอียง

- ฝึกต่อกันเป็นลำดับ โมเดลถัดไปแก้ที่ตัวก่อนทำผิด
- ตัวอย่าง XGBoost, LightGBM, AdaBoost
- แรงที่สุดกับข้อมูลตาราง แต่ overfit ได้ถ้าฝึกนานเกินไป



3. Stacking — ให้อีกโมเดลเรียนรู้วิธีรวม

- เอาคำทำนายของหลายโมเดลเป็นอินพุตให้โมเดลชั้นบน (meta-learner)
- ต้องใช้คำทำนายแบบ out-of-fold ไม่จั่นข้อมูลรั่ว
- ได้ผลดีเมื่อโมเดลแต่ละตัวเก่งคนละแบบ



4. Voting / Averaging — ทำง่ายที่สุด

- รวมผลจากโมเดลเดิมที่ฝึกด้วย seed ต่างกัน หรือคนละสถาปัตยกรรม
- จำแนกใช้โหวตเสียงข้างมากหรือเฉลี่ยความน่าจะเป็น ถดถอยใช้เฉลี่ย
- ต้นทุนต่ำสุด เหมาะเป็นด่านแรกของโปรเจกต์



ยาที่รักษา overfitting จะทำให้ underfitting แย่ลง และกลับกัน

ถ้าเป็น Overfitting

- เพิ่มข้อมูลและ augmentation
- เพิ่ม regularization และ dropout
- ลดความซับซ้อนของโมเดล
- หยุดฝึกเร็วขึ้น (early stopping)
- ลดจำนวน feature ที่ใส่เข้าไป
- ในงาน LLM — ลดจำนวนตัวอย่าง few-shot ที่เจาะจงเกินไป และเขียน prompt ให้เป็นกฎทั่วไป

ถ้าเป็น Underfitting

- เพิ่มขนาดหรือความลึกของโมเดล
- ลด regularization และ dropout ลง
- ฝึกนานขึ้น หรือเพิ่ม learning rate
- เพิ่ม feature ที่มีข้อมูลมากขึ้น
- ตรวจสอบว่า preprocessing ทำลายข้อมูลไปหรือเปล่า เช่น ลดความละเอียดภาพมากเกินไปจนตัวอักษรหาย
- ในงาน LLM — ใช้โมเดลที่ใหญ่ขึ้น หรือให้บริบทมากขึ้น